Python Activity-Based Learning

10 selected beginner-friendly case studies | Complete worked solutions

These solutions use sample inputs and explicitly stated assumptions because the assignment supplies topics rather than detailed numeric policies. Run each code example separately and capture its output for submission.

Case Study 12: Attendance Recovery Computation

1. Problem statement

Find the minimum consecutive attended classes needed to reach 75% attendance.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Attendance target is 75%; future classes are all attended; total classes must be positive.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
held = 40
attended = 26
target = 75
if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance data")
else:
    needed = 0
    while (attended + needed) * 100 < target * (held + needed):
        needed += 1
    print("Current attendance:", attended * 100 / held, "%")
    print("Extra classes needed:", needed)
    print("Final attendance:", round((attended + needed) * 100 / (held + needed), 2), "%")
```

8. Test cases

Test 1: held=40, attended=26 => 16 classes. Test 2: held=20, attended=16 => 0 classes. Test 3: attended>held => invalid.

9. Sample output (run result)

```
Current attendance: 65.0 %
Extra classes needed: 16
Final attendance: 75.0 %
```

10. Conclusion

A loop can find the minimum additional classes required for eligibility.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem statement

Simulate deposits and withdrawals, reject invalid operations, and show a transaction summary.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Starting balance is Rs. 1000; withdrawal limit is Rs. 500 per transaction; only positive amounts are allowed.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
balance = 1000
limit = 500
transactions = [("deposit", 300), ("withdraw", 400),
                ("withdraw", 700), ("deposit", -10)]
accepted = 0
rejected = 0
for kind, amount in transactions:
    if amount <= 0:
        print(kind, amount, "REJECTED: invalid amount")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        accepted += 1
        print("Deposited:", amount)
    elif kind == "withdraw" and amount <= limit and amount <= balance:
        balance -= amount
        accepted += 1
        print("Withdrawn:", amount)
    else:
        print(kind, amount, "REJECTED")
        rejected += 1
print("Final balance:", balance)
print("Accepted:", accepted, "Rejected:", rejected)
```

8. Test cases

Test 1: deposit 300 and withdraw 400 => Rs. 900. Test 2: withdraw 700 => rejected. Test 3: deposit -10 => rejected.

9. Sample output (run result)

```
Deposited: 300
Withdrawn: 400
withdraw 700 REJECTED
deposit -10 REJECTED: invalid amount
```

```
Final balance: 900
Accepted: 2 Rejected: 2
```

10. Conclusion

The simulation maintains a valid balance and reports rejected transactions.

Case Study 19: Multi-Product Shopping Budget Constraint

1. Problem statement

Compute a shopping bill and reduce quantities until it fits a budget.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Budget is Rs. 500; items are reduced from the last product backward; no discounts apply.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
items = ["Notebook", "Pen", "Bag"]
prices = [60, 20, 350]
quantities = [3, 4, 1]
budget = 500
total = 0
for i in range(len(items)):
    total += prices[i] * quantities[i]
print("Original total:", total)
for i in range(len(items) - 1, -1, -1):
    while total > budget and quantities[i] > 0:
        quantities[i] -= 1
        total -= prices[i]
print("Final total:", total)
for i in range(len(items)):
    print(items[i], "quantity:", quantities[i])
print("Within budget:", total <= budget)
```

8. Test cases

Test 1: budget=500 => final total Rs. 260 after removing bag. Test 2: budget=700 => no reductions. Test 3: budget=0 => all quantities 0.

9. Sample output (run result)

```
Original total: 610
Final total: 260
Notebook quantity: 3
Pen quantity: 4
Bag quantity: 0
Within budget: True
```

10. Conclusion

The list-based solution adjusts purchases to satisfy the budget.

Case Study 29: Password Policy Validation Engine

1. Problem statement

Validate a sample password and report each rule it fails.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Minimum 8 characters; at least one uppercase letter, lowercase letter, and digit; no spaces, triple repeated character, or substring password.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
pwd = "Hello123"
errors = []
if len(pwd) < 8:
    errors.append("At least 8 characters required")
if not any(c.isupper() for c in pwd):
    errors.append("Uppercase letter required")
if not any(c.islower() for c in pwd):
    errors.append("Lowercase letter required")
if not any(c.isdigit() for c in pwd):
    errors.append("Digit required")
if " " in pwd:
    errors.append("Spaces not allowed")
if "password" in pwd.lower():
    errors.append("Prohibited word used")
for i in range(len(pwd) - 2):
    if pwd[i] == pwd[i + 1] == pwd[i + 2]:
        errors.append("Three consecutive identical characters")
        break
if len(errors) == 0:
    print("Valid password")
else:
    print("Invalid password")
    for error in errors:
        print("-", error)
```

8. Test cases

Test 1: Hello123 => valid. Test 2: abc => length, uppercase and digit failures. Test 3: Password123 => prohibited word.

9. Sample output (run result)

```
Valid password
```

10. Conclusion

String checks can validate a password against stated rules.

Case Study 31: String-Based Examination Answer Evaluation

1. Problem statement

Compare student answers with an answer key and calculate scores with negative marking.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Correct answer +4; incorrect answer -1; unanswered answer marked - earns 0; both strings have equal length.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
key = "ABCDAB"
answers = ["AB-DAC", "ABCDAB"]
for number, answer in enumerate(answers, 1):
    if len(answer) != len(key):
        print("Student", number, "invalid answer length")
        continue
    correct = wrong = blank = 0
    for i in range(len(key)):
        if answer[i] == "-":
            blank += 1
        elif answer[i] == key[i]:
            correct += 1
        else:
            wrong += 1
    score = correct * 4 - wrong
    print("Student", number, "Correct:", correct,
          "Wrong:", wrong, "Blank:", blank, "Score:", score)
```

8. Test cases

Test 1: AB-DAC => 3 correct, 2 wrong, 1 blank, score 10. Test 2: ABCDAB => 6 correct, score 24.

9. Sample output (run result)

```
Student 1 Correct: 4 Wrong: 1 Blank: 1 Score: 15
Student 2 Correct: 6 Wrong: 0 Blank: 0 Score: 24
```

10. Conclusion

String comparison makes answer evaluation quick and consistent.

Case Study 35: Text Compression Using Character Runs

1. Problem statement

Compress consecutive repeated characters and compare encoded length with original length.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Use run-length encoding: character followed by count; retain original if encoding is not shorter.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
text = "aaabbcccc"
encoded = ""
i = 0
while i < len(text):
    count = 1
    while i + count < len(text) and text[i] == text[i + count]:
        count += 1
    encoded += text[i] + str(count)
    i += count
print("Original:", text, "Length:", len(text))
print("Encoded:", encoded, "Length:", len(encoded))
if len(encoded) < len(text):
    print("Stored:", encoded)
else:
    print("Stored:", text)
```

8. Test cases

Test 1: aaabbcccc => a3b2c4, shorter. Test 2: abc => original retained. Test 3: empty string => empty retained.

9. Sample output (run result)

```
Original: aaabbcccc Length: 9
Encoded: a3b2c4 Length: 6
Stored: a3b2c4
```

10. Conclusion

Run-length encoding helps when the text has repeated consecutive characters.

Case Study 36: Repeated Word and Phrase Detection

1. Problem statement

Count repeated words in a paragraph and show their word positions.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Words are separated by spaces; punctuation . , ! ? is removed; counting ignores case.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
paragraph = "Python is easy and Python is fun."
words = paragraph.lower().replace(".", "").replace(",", "")
words = words.replace("!", "").replace("?", "").split()
checked = []
for word in words:
    if word not in checked:
        positions = []
        for i in range(len(words)):
            if words[i] == word:
                positions.append(i + 1)
        if len(positions) > 1:
            print(word, "count:", len(positions), "positions:", positions)
        checked.append(word)
```

8. Test cases

Test 1: sample paragraph => python at 1,5; is at 2,6. Test 2: unique words => no repeated words.

9. Sample output (run result)

```
python count: 2 positions: [1, 5]
is count: 2 positions: [2, 6]
```

10. Conclusion

Lists and string operations identify repeated words and their locations.

Case Study 51: Bus Passenger Occupancy Simulation

1. Problem statement

Track boarding and alighting at each bus stop and detect invalid or over-capacity counts.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Bus capacity is 40; initial passengers 10; an invalid stop is rejected without changing the count.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
capacity = 40
passengers = 10
stops = [(15, 3), (25, 2), (5, 20), (3, 8)]
for number, (boarding, alighting) in enumerate(stops, 1):
    new_count = passengers + boarding - alighting
    if boarding < 0 or alighting < 0 or alighting > passengers:
        print("Stop", number, "invalid boarding/alighting")
    elif new_count > capacity:
        print("Stop", number, "capacity exceeded; stop rejected")
    else:
        passengers = new_count
        print("Stop", number, "passengers:", passengers)
print("Final passengers:", passengers)
```

8. Test cases

Test 1: stop 1 => 22 passengers. Test 2: stop 2 => capacity exceeded, rejected. Test 3: alighting more than current => invalid.

9. Sample output (run result)

```
Stop 1 passengers: 22
Stop 2 capacity exceeded; stop rejected
Stop 3 passengers: 7
Stop 4 invalid boarding/alighting
Final passengers: 7
```

10. Conclusion

The simulation flags capacity problems and keeps a valid passenger count.

Case Study 53: Food Waste Monitoring System

1. Problem statement

Calculate daily leftover meal percentages and flag excessive waste.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Prepared meals must be positive; leftovers = prepared - served; waste above 10% is excessive.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
prepared = [100, 120, 90]
served = [85, 110, 70]
for i in range(len(prepared)):
    if prepared[i] <= 0 or served[i] < 0 or served[i] > prepared[i]:
        print("Day", i + 1, "invalid data")
        continue
    leftover = prepared[i] - served[i]
    percent = leftover * 100 / prepared[i]
    print("Day", i + 1, "Leftover:", leftover,
          "Waste:", round(percent, 2), "%")
    if percent > 10:
        print("Excessive waste")
    else:
        print("Acceptable waste")
```

8. Test cases

Test 1: day 1 => 15% excessive. Test 2: day 2 => 8.33% acceptable. Test 3: day 3 => 22.22% excessive.

9. Sample output (run result)

```
Day 1 Leftover: 15 Waste: 15.0 %
Excessive waste
Day 2 Leftover: 10 Waste: 8.33 %
Acceptable waste
Day 3 Leftover: 20 Waste: 22.22 %
Excessive waste
```

10. Conclusion

The program identifies days needing improved meal planning.

Case Study 63: Multi-Product Price Fluctuation Analysis

1. Problem statement

Calculate percentage price changes and identify products with unusually large changes.

2. Objectives

Accept or define the required data, validate inputs, perform the calculations, and display a clear result.

3. Assumptions

Prices must be positive; threshold is 10% absolute change; use first and last historical prices.

4. Algorithm

1. Start. 2. Initialize sample data and policy values. 3. Validate inputs. 4. Apply loops and conditional checks to calculate results. 5. Display results and stop.

5. Flowchart

```
[START] --> [Set data] --> [Validate] --> [Process using loops / decisions] --> [Display] --> [END]
```

6. Pseudocode

```
BEGIN
  SET sample inputs and rules
  FOR each applicable item / WHILE target not reached
    IF inputs or constraints invalid THEN report error
    ELSE compute result and update values
  END LOOP
  DISPLAY final results
END
```

7. Python source code

```
products = ["Rice", "Oil", "Sugar"]
history = [[50, 52, 60], [100, 95, 90], [40, 41, 42]]
threshold = 10
largest = -1
largest_product = ""
for i in range(len(products)):
    old = history[i][0]
    new = history[i][-1]
    if old <= 0 or new <= 0:
        print(products[i], "invalid price")
        continue
    change = (new - old) * 100 / old
    print(products[i], "Change:", round(change, 2), "%")
    if abs(change) > threshold:
        print("Above fluctuation threshold")
    if abs(change) > largest:
        largest = abs(change)
        largest_product = products[i]
print("Largest fluctuation:", largest_product, round(largest, 2), "%")
```

8. Test cases

Test 1: Rice => +20%, flagged. Test 2: Oil => -10%, not flagged (strictly above). Test 3: Sugar => +5%, not flagged.

9. Sample output (run result)

```
Rice Change: 20.0 %
Above fluctuation threshold
Oil Change: -10.0 %
Sugar Change: 5.0 %
Largest fluctuation: Rice 20.0 %
```

10. Conclusion

The program compares historical prices and highlights large changes.